Scalable web app on DigitalOcean Kubernetes

Technical Account Manager case study

Pooja Singh | October 30, 2026

A small Node.js application with load distribution, pod autoscaling, a managed HA control plane, and a measured customer handoff.

Deliverable	Included
Architecture	Illustrative scaled state; health-check and availability boundaries.
Setup and operations	Deployment, safe HPA migration, verification, troubleshooting, teardown.
Cost and performance	Explicit pricing assumptions and raw-backed endpoint benchmarks.
Quarterly business review	Two-page customer review with risks, owners, and next actions.
Evidence	Raw API/benchmark outputs, versions, tests, and dashboard screenshots.

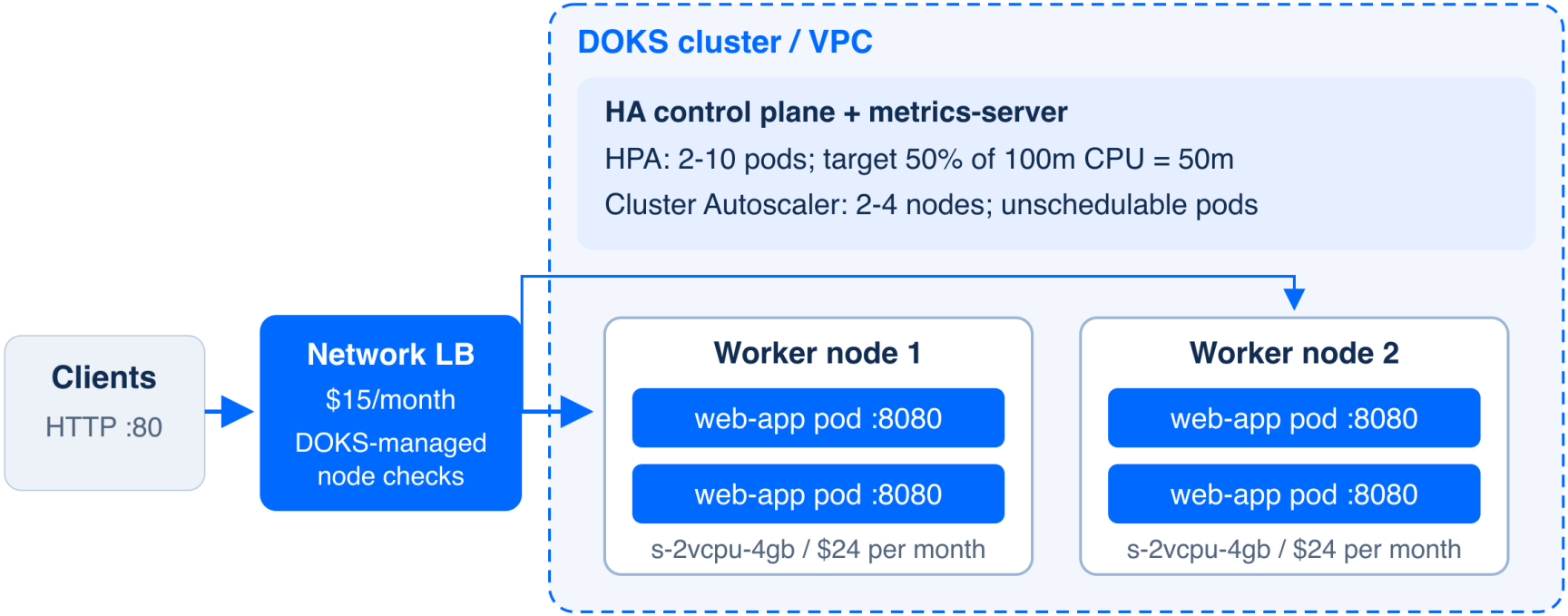
Source repository

github.com/pooja1909/do-k8s-scalable-webapp

Deployed release: 1.0.2. Public demo: http://137.184.244.5. The evidence includes endpoint benchmarks, a monitored rolling deployment, and controlled voluntary pod recovery.

Scalable web application on DOKS

Illustrative scaled state: 4 app pods shown. Baseline: 2 pods. Region: sfo3.



Availability design: pod probes, graceful shutdown, rolling updates, and voluntary-eviction PDB.

Spread rules act during scheduling; surviving pods can share a node after scale-down.

Developer → DOCR Starter (image 1.0.2) → nodes pull image → safe deployment script

Modeled subtotal: $103 baseline / $151 at 4 nodes. Overages and added services excluded.

HA: $40/month. The 99.95% SLA applies to the control plane only, not application availability.

Setup and Operations Guide

Prepared for the customer's engineering team. Allow about 45 minutes for first-time setup; provisioning and autoscaling times vary.

1. Architecture and prerequisites

The Node.js app listens on port 8080. A Service exposes HTTP traffic on port 80 through a regional network Load Balancer. DOKS manages node health checks; Kubernetes uses `/healthz` for liveness and `/readyz` for readiness. The HPA owns the app replica count, targeting 50% of the 100m CPU request. Cluster Autoscaler adds nodes when eligible pods cannot be scheduled, within a 2-4 node pool.

Install `doctl`, `kubectl`, Docker with buildx, Python 3, and optionally `hey`. On macOS: `brew install doctl kubectl hey`; install Docker Desktop separately. Authenticate with `doctl auth init`. Never put API tokens in this repository.

Observed on October 30, 2026: **doctl 2.1.0**, **kubectl v1.36.1**, **DOKS 1.37.0-do.2**. Local regression tests used Node **v24.22.1**, plus the Node 22 Alpine linux/amd64 container. See `evidence/local/` and `evidence/final/baseline/versions.json` for exact output. These are observed versions; check availability before creating a new cluster.

2. Configure and create

```
git clone https://github.com/pooja1909/do-k8s-scalable-webapp.git
cd do-k8s-scalable-webapp
```

Edit `scripts/config.env`: registry name, region, cluster/pool names, node size and bounds, image tag, and Kubernetes version. The sample registry is `pooja-doks-demo-2026`, matching the existing demo. Use a unique registry for another account. Check available versions with `doctl kubernetes options versions` before new creation.

```
./scripts/01-create-infra.sh
```

The creation script explicitly uses `--ha=true`, the selected `--version`, a 2-4 node autoscaling pool, and the metrics-server add-on. It verifies an existing registry if creation fails, connects registry access, saves kubeconfig, and waits for metrics. It returns failure if metrics remain unavailable. This step creates billable resources; skip it when updating the existing cluster.

3. Build and deploy

```
./scripts/02-build-push.sh
kubectl config current-context
./scripts/03-deploy.sh
```

Buildx targets `linux/amd64`. Use a unique image tag for each release; this submission uses **1.0.2**. Deployment substitutes the same tag into `APP_VERSION`. The deployment script verifies context `do-<region>-<cluster>` and matches its API endpoint against the DigitalOcean cluster before making changes.

It applies the namespace and HPA, safely migrates replica ownership, applies the app, Service, and PDB, waits for rollout and at least two available pods, then prints the Load Balancer URL. The Service explicitly selects `REGIONAL_NETWORK` with TCP forwarding and leaves health checks under DOKS control.

HPA ownership of replicas

The HPA owns the live replica count. If a saved client-side apply configuration contains `spec.replicas`, removing that field from the manifest alone can reset the live Deployment to one replica. The script updates `kubectl.kubernetes.io/last-applied-configuration`, removing **only** `spec.replicas`, before applying the replica-free manifest. Other saved fields stay intact so ordinary deletions continue to work. Deployments therefore preserve the HPA's live replica count.

This automates the Kubernetes migration procedure. A fresh Deployment may start with the API default of one until HPA reconciliation; the script does not announce readiness until two are available. Do not use `kubectl scale` or remove the live replica field during migration.

4. Verify and collect evidence

```
kubectl -n web-app get pods -o wide
kubectl -n web-app get hpa web-app
kubectl -n web-app get svc web-app
kubectl top nodes
./scripts/demo-load-balancing.sh
./scripts/capture-evidence.sh baseline
./scripts/benchmark.sh light
./scripts/benchmark.sh work
```

Run the benchmarks sequentially without internal load-generator traffic. The script requires two ready replicas and an idle HPA before each run and verifies the live app version. Defaults: `hey -z 60s -c 50 -disable-keepalive`; the work test uses `/work?ms=50`. Fresh connections allow new pods to receive new traffic. Set `KEEP_ALIVE=true` for a separate persistent-connection stress test. Each test saves raw output, command, image/digest, versions, and HPA/pod/node snapshots. Recovery-folder numbers count sleep intervals; use recorded timestamps for elapsed time.

For screenshots, capture the cluster HA and node-pool settings and the Load Balancer health-check settings in the DigitalOcean control panel, record capture time, and keep them alongside the raw API snapshots. Do not label a diagram or reconstructed display as a dashboard screenshot.

5. Operate and troubleshoot

Task or symptom	Action
New version	Change `IMAGE_TAG`, build/push, then run `03-deploy.sh`. Record errors/latency throughout a rollout before claiming no downtime.
Rollback	`kubectl -n web-app rollout undo deployment/web-app`; then align source/tag with the rollback before the next deploy.
HPA unknown	Check `kubectl top pods -n web-app`, CPU requests, and `kubectl -n web-app describe hpa web-app`.
Pods pending	Inspect scheduler events, allocatable resources, taints, spread constraints, pool max and account limits. High CPU alone does not trigger node scaling.
Image pull failure	Verify registry name/tag and cluster registry integration with `doctl kubernetes cluster registry add <cluster>`.
Wrong image architecture	Rebuild using `--platform linux/amd64`.
LB unhealthy	Inspect effective LB config with `doctl compute load-balancer get <id> -o json`, Service annotations, endpoint slices, node health, and pod readiness. Do not assume the LB is probing the app.
Pods share a node after scale-down	Check both workers are Ready and schedulable. Run the monitored pod-recovery procedure below, then verify one ready replica per worker. Evaluate automated rebalancing for production.

At an idle two-pod baseline, run `python3 scripts/reliability-check.py pod-recovery` to inspect the plan, then add `--execute` to evict one pod through the PDB. The script logs HTTP samples at up to 10 requests/second and cluster state every two seconds, waits for two ready replicas, and records 20 seconds of post-recovery traffic. It never stops a worker. For a release, use the `rollout --execute` action to monitor the deployment script. Each action preserves its evidence and refuses to overwrite an existing run directory.

Topology constraints act when pods are scheduled; they do not rebalance existing pods. The PDB limits voluntary evictions only. `maxUnavailable: 0` is a rollout setting, not proof of application uptime. HA's 99.95% SLA covers the control plane only.

6. Teardown and billing review

```
./scripts/99-teardown.sh
./scripts/99-teardown.sh --execute
# Only for a registry dedicated to this demo:
./scripts/99-teardown.sh --execute --delete-registry
```

The default prints a plan and namespace inventory. Execution requires a typed cluster ID. The script verifies both context and cluster endpoint, deletes the app namespace before the cluster, stops on errors, and does not use `--dangerous`. Registry deletion is opt-in, requires exact name verification and separate confirmation, and retains doctl's own prompt. Check remaining LBs, volumes, snapshots, and the billing dashboard; successful cluster deletion alone is not proof that all charges stopped.

Recommend a **$120 billing alert**, subject to the customer's agreed account-wide budget, plus CPU, HPA metric, readiness, latency, and error alerts. An alert does not stop spending.

Cost and Performance Analysis

Review date: October 30, 2026. Prices are USD list-price inputs; projections exclude tax, discounts, bandwidth overages, and additional services. Performance observations are short demo tests, not a production SLO or availability measurement.

1. Infrastructure pricing

The deployment uses a **REGIONAL_NETWORK** Load Balancer with one node, at **$15/month**, verified by API and dashboard. Sources: LB pricing, Droplet pricing, and DOKS pricing.

Component	Configuration	Monthly input
Workers	Basic `s-2vcpu-4gb`, 2 vCPU / 4 GiB each; 2-4 nodes	$24 each; $0.03571/hour equivalent
Control plane	HA enabled; explicitly selected by creation script	$40
Load Balancer	Regional network, one node	$15
Registry	Starter tier assumption; one repository within free allowance	$0

HA was verified by API and screenshot. Its 99.95% SLA covers the control plane only; it does not establish application availability. The registry model assumes usage stays within its free-tier allowance.

2. Illustrative monthly scenarios

Scenario	Calculation	Infrastructure subtotal
Baseline: two continuously running workers	2 x $24 + $40 + $15	**$103.00**
Assumed daily peak: two extra workers for 6 h/day, 30 days	$103 + 2 x 6 x 30 x $0.03571	**$115.86**
Four workers continuously; static peak sizing comparison	4 x $24 + $40 + $15	**$151.00**

The illustrative daily-peak scenario saves **$35.14, or 23.3%**, against $151 static peak sizing. No billing history or observed node scale-out supports this duty cycle: it is a planning assumption. Bundled worker plans have a monthly cap and per-second billing; the hourly equivalent is used for transparent estimation. Reconcile actual usage and rounding against the invoice.

The subtotal is a hard spending ceiling. Pod and node limits bound configured capacity; bandwidth, additional LB nodes, storage, taxes, other services, and other account resources can increase the bill. Recommend a **$120 billing alert** aligned with the agreed account budget. An alert does not stop spending.

3. Endpoint benchmark evidence

Both tests used `hey -z 60s -c 50 -disable-keepalive` from the same laptop through the public Load Balancer on October 30, 2026 (PDT), running release **1.0.2**. Each began with **two ready replicas and an idle HPA**, without internal load-generator traffic. Raw outputs and timestamped before/during/after/recovery snapshots are in `evidence/final/light/` and `evidence/final/work/`.

Measurement	**Light:** `/api/info`	**CPU-heavy:** `/work?ms=50`
Completed duration	71.7956 seconds	62.9169 seconds
Requests/second	201.20	36.87
p50 / p95 / p99	168.6 / 389.7 / 507.3 ms	797.2 / 3,206.0 / 3,687.9 ms
HTTP results	14,443 responses, all 200	2,299 responses, all 200
Reported request errors	2 timeouts / 14,445 attempts (0.014%)	21 timeouts / 2,320 attempts (0.91%)
Starting replicas	2	2
Replica response, including recovery	2 to 5, then back to 2	2 to 10, then back to 2
Observed worker nodes	2 throughout	2 throughout

The endpoints model different workloads: `/api/info` returns small JSON, while `/work?ms=50` deliberately consumes CPU in 5-ms chunks, yielding to I/O between chunks. Results include network latency and the autoscaling ramp. Fresh TCP connections let new pods receive new traffic. The 60-second setting controls request generation; completion can extend the reported duration. Latency percentiles describe completed responses; errors are listed separately. These are demo measurements, not production SLO validation. Node autoscaling is **configured, not exercised**.

A separate **keep-alive CPU stress test** (`persistent-work/`) recorded **1,289 HTTP 200 responses and one timeout**: 1/1,290 attempts, or **0.078%**. It ran for 77.1167 seconds including completion, at 16.73 requests/second; p95 was **3,327.6 ms** and p99 **9,126.0 ms**. Sampled pods remained ready with no restarts. The fresh-connection CPU test also had 21 timeouts (0.91% of attempts). Neither CPU test establishes a production latency or error SLO. The differing connection modes are not directly comparable. Uneven pod CPU is consistent with connection concentration, but the client-observed timeouts were not isolated to a specific application, client/network, or platform cause. This result is retained as a production-sizing risk.

The separate reliability checks recorded **485/485** HTTP 200 samples during deployment and **361/361** during voluntary pod recovery. Two ready replicas were observed **7.61 seconds** after the pod-eviction action started. The reliability section explains sampling, scope, and limitations; no application-uptime percentage or sudden-node-failure result is claimed.

4. Why CPU pressure did not trigger more nodes

Each app pod requests **100m** and is limited to **500m**. The HPA's 50% utilization target is **50m per pod**, not 50% of a worker CPU or of the container limit. Ten pods request only **1 CPU** in total, yet their limits allow up to **5 CPUs** across the two workers' **4 nominal vCPUs**, before system reservations. This explains how the scheduler can consider pods to fit while actual CPU pressure is high.

The Cluster Autoscaler responds to unschedulable pods and scheduling constraints; high observed CPU alone is not a scale-out trigger. Node count remained at two throughout the measured tests; successful node scale-out is not claimed.

Right-size requests and retune HPA **together**, then rerun a controlled test from a documented baseline. For example, changing 100m to 250m while keeping 50% changes the effective target from 50m to 125m. Preserving a 50m target would require 20%, but neither 250m nor 20% is a production recommendation without measurement. Record throttling, CPU demand, latency/errors, pending-pod reasons, scale-out time, and recovery before selecting values. Do not increase the CPU request in isolation and claim improved behavior.

5. Availability and cost tradeoffs

Two smaller nodes offer more placement options than a single larger node, but failover depends on placement and available capacity. The final baseline has one ready app pod on each worker; placement must be checked after every scale-down. Topology spread applies at scheduling time; increasing the minimum to three replicas does not guarantee balanced survivors after scale-down. A PDB constrains voluntary evictions and cannot prevent involuntary node loss.

Keep the Basic worker plan for the demo and evaluate sustained CPU needs with representative traffic before choosing dedicated CPU. Keep HTTPS, deployment automation, CDN, and managed databases on the prioritized roadmap. Use representative traffic, recovery exercises, and billing history to validate production performance, availability, and savings.

Reliability Validation

Measured October 30, 2026, through the public Load Balancer. The application has two baseline replicas, readiness/liveness probes, a 30-second termination grace period, graceful connection draining, a rolling strategy with zero unavailable replicas, and a PDB requiring one available replica. Scheduling spreads pods across workers. CPU work yields to I/O between 5-ms chunks so probes can respond under concurrent work.

Timed deployment and recovery checks

Measurement	Rolling deployment	Voluntary pod recovery
HTTP sampling window (UTC)	22:51:47.100-22:52:37.409	22:52:58.957-22:53:36.491
Samples / HTTP 200 / errors	485 / 485 / 0	361 / 361 / 0
HTTP p95 / maximum latency	107.4 / 168.2 ms	111.1 / 120.8 ms
Minimum sampled available replicas	2	1
Action start to two ready replicas observed	20.37 seconds	7.61 seconds
Placement after the action	One ready replica per worker	One ready replica per worker

The rollout deployed release **1.0.2** and preserved the HPA's desired count of two throughout sampled observations. Its saved apply configuration contains no replica count. The recovery check evicted one app pod through the `policy/v1` Eviction API with a UID precondition and PDB enforcement, then observed its replacement become ready. The worker remained online. Public responses came from pods on both workers in each test.

Reproducibility and interpretation

`scripts/reliability-check.py` verifies the cluster identity and requires two healthy replicas, a disruption allowance, and no internal load generator. It records ten seconds of baseline traffic, performs the action, waits for recovery, then records twenty seconds of additional traffic. HTTP sampling is sequential at up to ten requests/second, with a three-second request timeout. Cluster state is sampled approximately every two seconds.

Raw HTTP, pod, HPA, PDB, action, and timing logs are under `evidence/final/rollout/` and `evidence/final/pod-recovery/`. These deployment/recovery windows had **no observed request errors**; the performance section separately reports the CPU stress-test timeouts. Gaps between samples are unmeasured; these short tests do not establish a long-term uptime percentage, production SLO, or recovery from sudden worker or regional failure.

Operational follow-through

After autoscaling, check actual placement. Topology spread governs scheduling and does not rebalance survivors after scale-down. At an idle baseline, a monitored PDB-respecting eviction can restore distribution when both workers are eligible; verify the result. For production, alert on missing ready replicas and concentrated placement, and evaluate automated rebalancing under an explicit eviction policy.

Agree customer recovery and latency/error targets, then test abrupt worker loss, sustained load, and rollback in a controlled window. HA control-plane configuration and voluntary-disruption protection support availability; application failure tolerance still requires workload-specific testing.

Quarterly Business Review

Web platform on DigitalOcean Kubernetes

Prepared by: Pooja Singh, Technical Account Manager

Audience: CTO, VP Engineering, platform team | **Review:** October 30, 2026

Customer outcome and current infrastructure

The deployed Node.js application demonstrates public load distribution, CPU-driven pod autoscaling, a monitored rolling deployment, and recovery from a voluntary pod eviction. The environment uses DOKS in **sfo3**, an HA control plane, two **2-vCPU / 4-GiB** Basic workers, and a **regional network Load Balancer**. HPA bounds are **2-10 pods**; the node pool is configured for **2-4 workers**. Release **1.0.2** is live and its source hashes match the submitted code.

Customer goal	Measured position
Scalability	Both endpoint tests began with two ready replicas and an idle HPA. Light scaled 2 to 5 to 2; CPU scaled 2 to 10 to 2, including recovery. Workers stayed at two; node scale-out was not exercised.
Performance	Light endpoint: p95 389.7 ms at 201.20 requests/sec, with two timeouts (0.014%). CPU-heavy endpoint: p95 3,206.0 ms at 36.87 requests/sec. Both used 50 concurrent connections for 60 seconds. The CPU test had 21 timeouts in 2,320 attempts (0.91%).
Reliability	Rollout: 485/485 sampled HTTP 200 responses. Pod recovery: 361/361 HTTP 200 responses; two ready replicas observed after 7.61 seconds. Final baseline has one ready replica per worker.
Cost	Modeled $103 baseline, $115.86 with two extra workers for an assumed 6 hours/day, and $151 for four continuously running workers. These are infrastructure projections.

Business interpretation

The additional keep-alive CPU stress test recorded one timeout in 1,290 attempts (0.078%), with p95 3.33 seconds. Pods stayed ready in sampled observations. Representative connection patterns and capacity need validation before production.

Light-response p95 was below 0.4 seconds; the observed timeouts require follow-up before production. The CPU-heavy endpoint deliberately consumes CPU while yielding to I/O between short chunks, making capacity and tail latency visible. At ten pods, requests total one CPU while limits total five CPUs across four nominal worker vCPUs. High utilization can coexist with sufficient scheduling reservations. Cluster Autoscaler needs unschedulable pods; resource requests and HPA targets should be tuned together against representative traffic.

The network Load Balancer costs **$15/month**. The illustrative daily-peak scenario saves **$35.14 (23.3%)** against static four-worker sizing; the assumed duty cycle is not measured billing history. HA costs $40/month, and its 99.95% SLA covers the control plane. Short reliability checks establish observed outcomes within their sampling windows, not a long-term application uptime percentage.

Prioritized customer action plan

When / owner	Action and acceptance evidence
Before launch / platform + TAM	Agree p95, error-rate, and recovery objectives. Test representative traffic and sustained CPU load; tune CPU requests and HPA together. Record throttling, pending reasons, capacity arrival, and cost impact.
Before launch / platform	Exercise abrupt worker loss and rollback with timed HTTP monitoring. Confirm placement and headroom after recovery; compare measured recovery time with the agreed objective.
Within 2 weeks / TAM + platform	Configure latency, error, readiness, HPA-metrics, and placement alerts. Agree the account budget and configure a $120 billing alert if appropriate.
Within 30 days / platform	Add HTTPS with TLS termination at an ingress/app compatible with the network LB. Add CI/CD with immutable images, automated checks, and tested rollback.
Next quarter / CTO + platform	Review two weeks of workload and billing data. Evaluate dedicated CPU, request-rate scaling, CDN for static assets, and Managed Databases when persistent data is introduced.

Risks and mitigations

Risk	Mitigation / decision
Uneven pod placement after scale-down	Inspect placement; use monitored PDB-respecting pod replacement when needed. Evaluate automated rebalancing with an eviction policy.
CPU saturation and slow capacity arrival	Maintain measured headroom and tune requests/HPA together. Lengthen and validate the 60-second demo scale-down window for production traffic.
Single region and untested abrupt worker loss	Define recovery objectives and test the failure runbook. Evaluate a second region when business requirements justify it.
Spending above the modeled subtotal	Review transfer, LB size, storage, and other account services. Capacity bounds and billing alerts do not cap spending.
Overinterpreting short reliability tests	PDB covers voluntary evictions. Require sustained monitoring and abrupt-failure tests before adopting an application availability target.

Next customer check-in: confirm performance and recovery objectives, account budget, and action owners. Review raw evidence together and prioritize production work by customer impact. HTTPS, CI/CD, CDN, and persistent storage are planned capabilities.

Evidence Index

Captured October 30, 2026 (PDT); raw timestamps use UTC. Deployed release **1.0.2** is served through the public Load Balancer. `final/image-verification.txt` records Node 22.23.4, the non-root user, and source hashes matching the repository.

Evidence	Location and scope
Endpoint benchmarks	`final/light/hey.txt` and `final/work/hey.txt`: fresh-connection tests, 50 workers, 60-second request window, two-pod baseline. Full results, including timeouts, appear in the performance section.
Keep-alive CPU stress	`final/persistent-work/`: 1,289 HTTP 200 responses, one timeout (0.078% of attempts), p95 3,327.6 ms.
Scaling and recovery	Each benchmark has before/during/after/recovery snapshots. `final/*placement-recovery/` records controlled redistribution after scale-down. Workers stayed at two.
Rolling deployment	`final/rollout/`: 485/485 HTTP 200 samples, zero observed request errors, minimum sampled available replicas two, desired replicas preserved at two.
Pod recovery	`final/pod-recovery/`: one PDB-respecting voluntary eviction, 361/361 HTTP 200 samples, two ready replicas observed 7.61 seconds after action start.
Final cluster state	`final/baseline/`: one ready app replica per worker; HA true; DOKS 1.37.0-do.2; doctl 2.1.0; kubectl v1.36.1.
Effective LB settings	`final/baseline/load-balancer.json`: regional network, one node, managed HTTP health checks on 10256 at `/healthz`, distinct from the app on 8080.
Dashboard screenshots	`screenshots/`: HA, node autoscaling, LB type/cost, and health checks. Capture times are in `captures.txt`.
Regression checks	`local/node24-tests.txt` and `local/docker-node22-tests.txt`: seven passing test cases each. `local/script-safety-tests.txt`: seven passing safety tests with mocked cloud commands.
Release verification	`final/release.json` and `final/live-smoke.json`: image digest/source hashes and live endpoint checks. `local/validation-summary.txt`: shell, Python, YAML, and manifest relationships.

`scripts/summarize-evidence.py` derives `final/summary.json`. Snapshot command logs preserve timestamps and exit statuses; pod JSON records image digests. Reliability logs contain every HTTP sample and sampled cluster state. Four optional API captures in light `recovery-30` timed out; stderr is retained. Recovery-folder numbers count sleep intervals, not wall time.

Evidence boundaries

The rollout and voluntary pod-recovery windows had no observed request errors. Sampling gaps, long-term uptime, sudden worker loss, regional failure, and node autoscaling remain unmeasured. The cost model is illustrative, with no invoice validating the assumed extra-node duty cycle.

Technical and pricing references

1. Horizontal Pod Autoscaling and replica migration

https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

2. Topology spread constraints

https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/

3. Pod disruptions and PDB boundaries

https://kubernetes.io/docs/concepts/workloads/pods/disruptions/

4. Service proxies and connection routing

https://kubernetes.io/docs/reference/networking/virtual-ips/

5. DOKS Load Balancer settings and managed health checks

https://docs.digitalocean.com/products/kubernetes/how-to/configure-load-balancers/

6. DOKS Cluster Autoscaler

https://docs.digitalocean.com/products/kubernetes/how-to/autoscale/

7. doctl cluster creation flags

https://docs.digitalocean.com/reference/doctl/reference/kubernetes/cluster/create/

8. DOKS pricing

https://docs.digitalocean.com/products/kubernetes/details/pricing/

9. Droplet pricing

https://www.digitalocean.com/pricing/droplets

10. Regional Load Balancer pricing

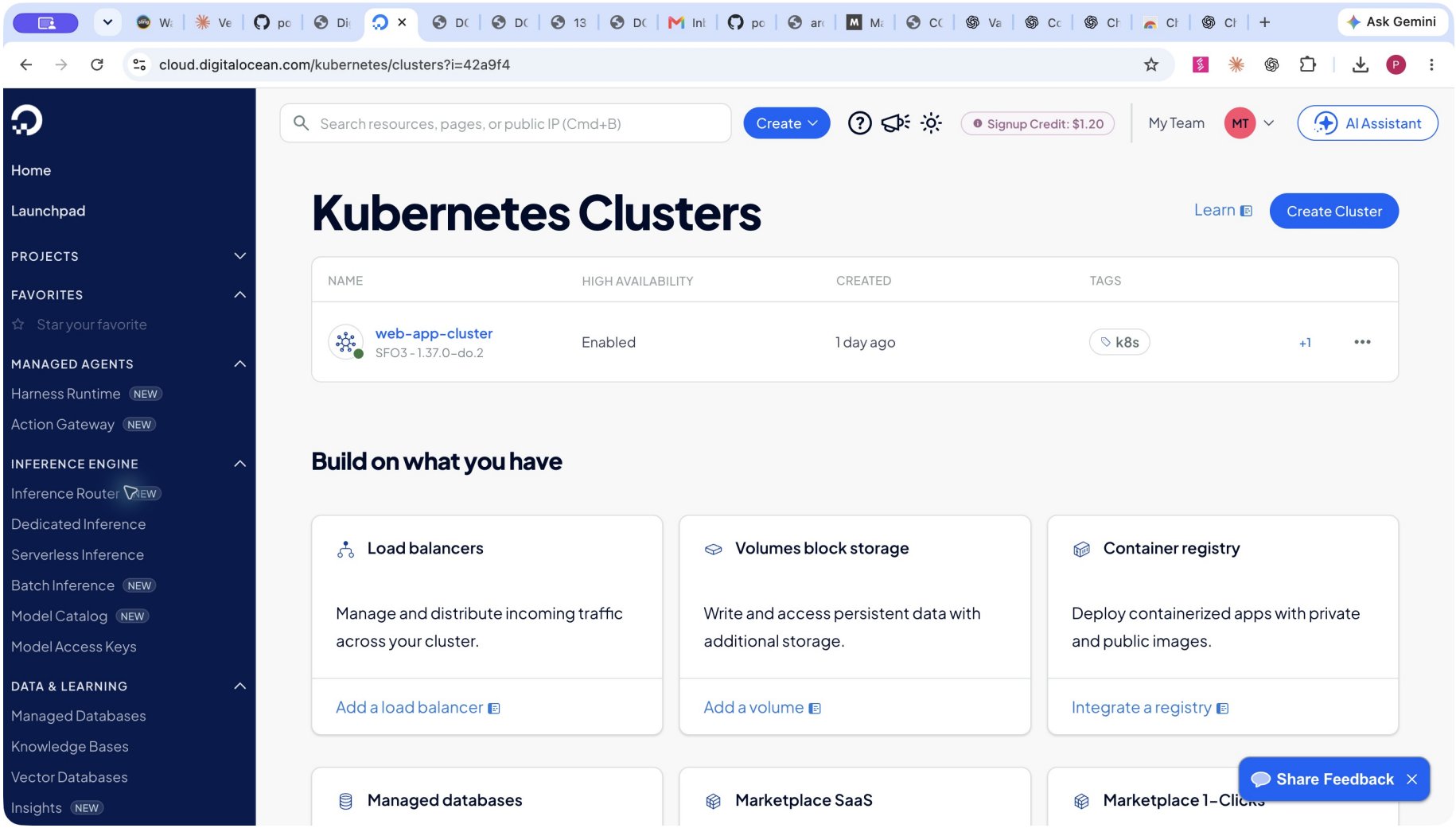
https://docs.digitalocean.com/products/networking/load-balancers/details/pricing/

11. HA control-plane SLA

https://www.digitalocean.com/sla/doks

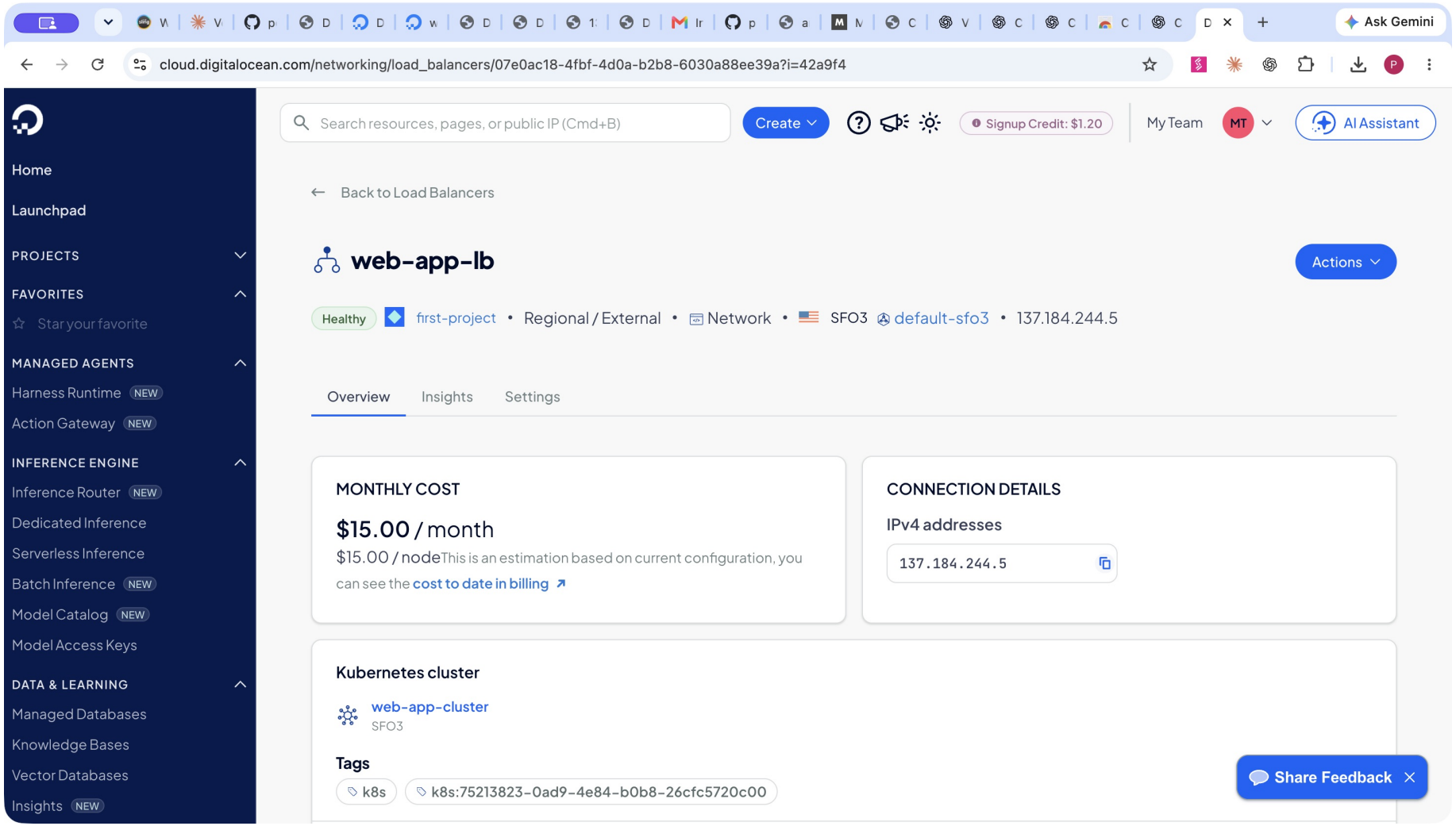
Official references checked October 30, 2026. The evidence folder contains the actual observations; configuration documentation is not a substitute for measured application availability.

Dashboard evidence / high availability



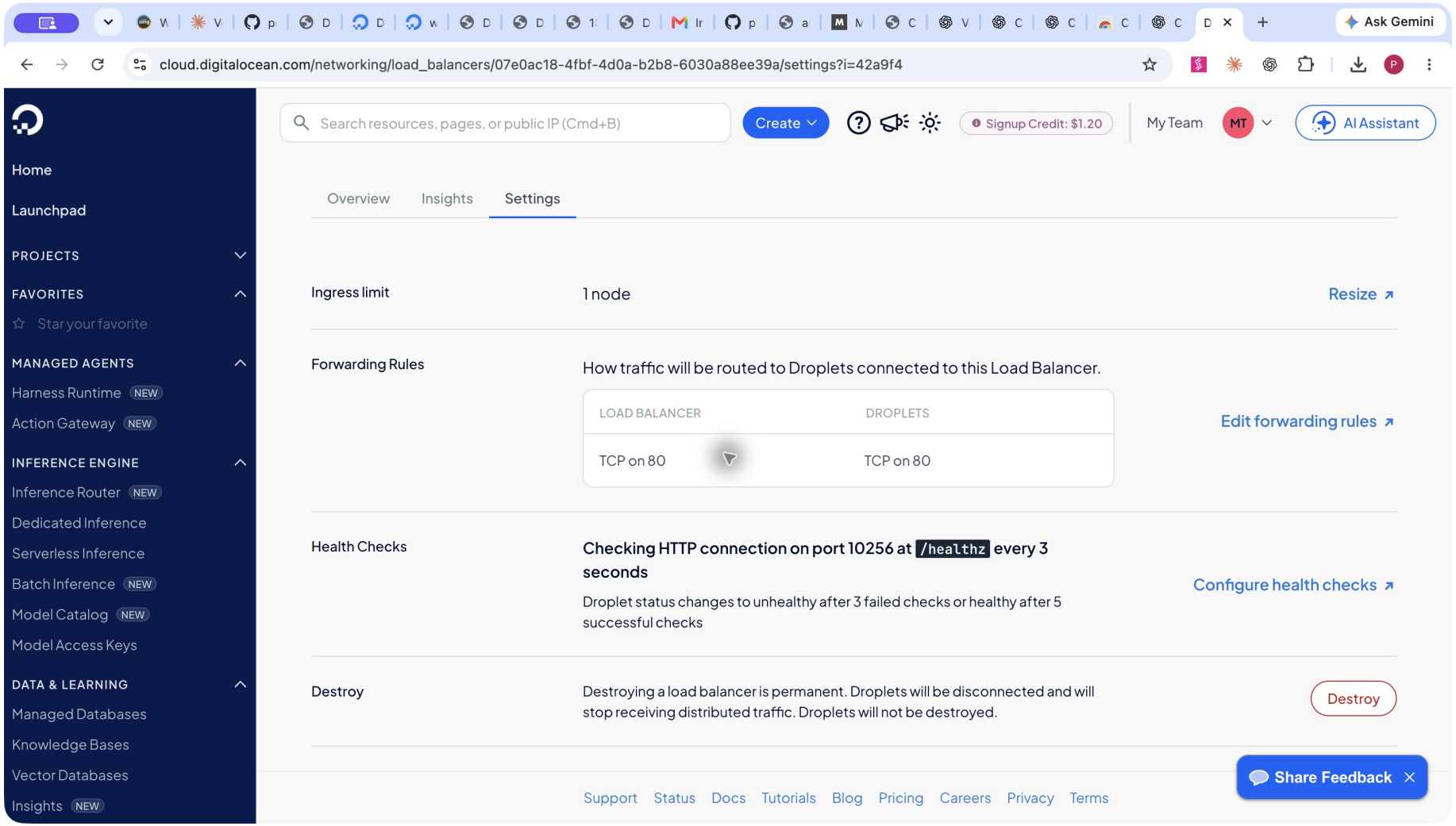
The cluster list shows HA Enabled and Kubernetes 1.37.0-do.2. This verifies configuration, not application uptime.

Dashboard evidence / Load Balancer type and cost



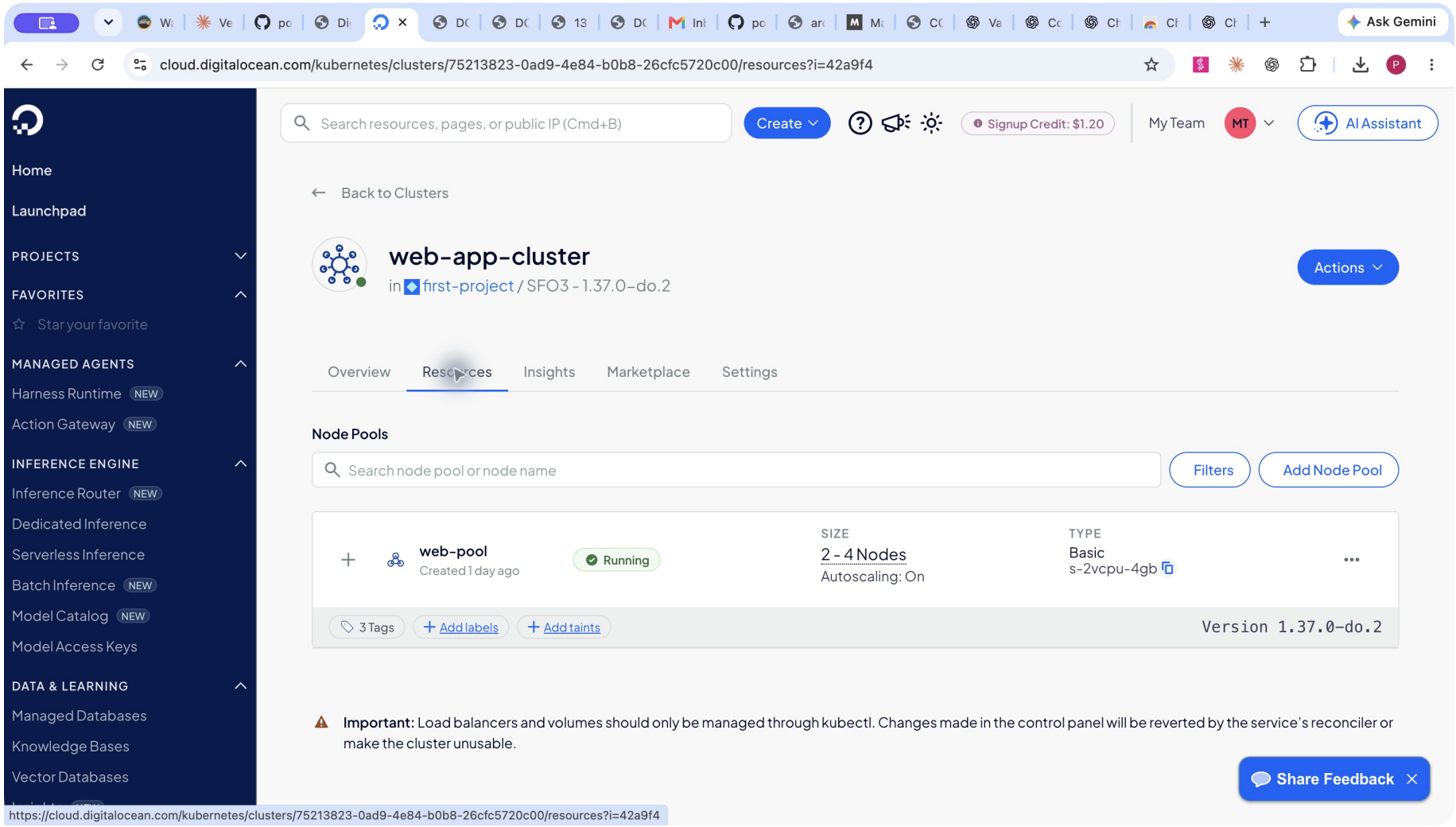
Regional / External Network Load Balancer; dashboard estimate $15 per month. This is the Load Balancer used in the cost model.

Dashboard evidence / effective health checks



HTTP on port 10256 at /healthz: the node health endpoint, distinct from the app on port 8080.

Dashboard evidence / node pool



Node pool autoscaling is enabled from 2 to 4 nodes. Actual node scale-out was not exercised in these benchmarks.